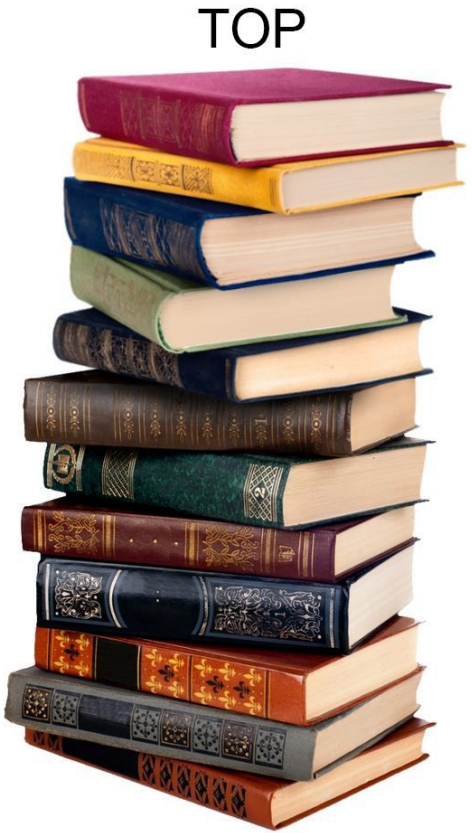
Динамическая память

STACK & HEAP

Стек

Что это?

Область в памяти, где живут локальные переменные, параметры функции, временные объекты.



Стек

Свойства стека

- Скорость максимальная – выделение/освобождение O(1), просто сдвиг указателя стека.
- Автоматическое управление – переменная умерла, когда вышли из области видимости
- Ограниченный размер (обычно 1 ~ 8 МБ)
- Организована по принципу LIFO (Last In First Out)

https://youtu.be/xnrx4L6B8KE?si=4wtEEt7gfd_s6Pi5



Функции на стеке

КАК РАБОТАЕТ СТЕК | ОСНОВЫ ПРОГРАММИРОВАНИЯ

Переполнение Стека (Stack Overflow)



Переполнение – это когда новый frame не влезает в оставшееся место

- Бесконечная или слишком глубокая рекурсия
  Каждый вызов функции создает новый frame – стек растет без остановки – overflow
- Огромные локальные массивы
- Слишком большая цепочка вызовов

Stack Overflow

Как именно это происходит технически (глубже)

1. Стек имеет **границы**, заданные ОС при запуске процесса
2. Указатель стека (stack pointer, SP) двигается при вызовах
3. При попытке SP выйти за пределы → аппаратный trap
4. ОС убивает процесс

На Linux/macOS это выглядит в логах как:

```
Segmentation fault (core dumped)
```

На Windows:

```
Stack overflow.
```

Куча

Куча (heap) – память “по запросу”

Это область в памяти, где объекты живут столько, сколько ты сам решишь.

Выделяется вручную через new, malloc (это в C)

Освобождается вручную через delete, delete[]

Куча

Свойства кучи

- Жить могут объекты любого размера
- Lifetime не привязан к области видимости
- Ограничен размерами оперативной памяти (RAM)
- Нет сборщика мусора (Garbage Collector) (В C++)
- Легко создать утечку памяти

Как работает new

new делает три вещи:

1. Считает нужный размер объекта
2. Выделяет память на куче
3. Вызывает конструктор объекта

Как работает delete

delete делает обратное:

1. Вызывает деструктор
2. Освобождает память

Важно: delete и delete[] – разные операции, и вызов неправильной может привести к UB.

delete должен знать размер объекта, delete[] – размер массива

Как выделить место на куче

```
int* p = new int(10);
delete p;

int* arr = new int[50];
delete[] arr;  // обязательно []
```

Частые ошибки

1. delete на не-массив

```
int* arr = new int[10];
delete arr;
```

2. Двойное удаление

```
int* p = new int(10);
delete p;
delete p;
```

Немного RAII

Что такое RAII

RAII (от англ. *Resource Acquisition Is Initialization*) — техника программирования в C++, которая **связывает жизненный цикл ресурса** (например: память, поток выполнения, открытый файл, заблокированный мьютекс, соединение с базой данных) с **жизненным циклом объекта**.

Иными словами:

- Когда объект создается — он захватывает ресурс.
- Когда объект уничтожается — он освобождает ресурс.
- Всё это происходит автоматически и безопасно.

Немного RAII

Почему это важно

- RAII гарантирует, что ресурс **доступен** любому коду, который использует объект. (То есть, объект-запрос ресурс — и ресурс готов к использованию.)
- RAII также гарантирует, что **все ресурсы будут освобождены**, когда закончится жизнь объекта, причем в обратном порядке захвата.
- Если при создании объекта ресурс захватить не удалось (например, файл не открылся или память не выделилась) — конструктор может выбросить исключение, и любые уже захваченные ресурсы будут освобождены автоматически через механизм “раскрутки стека” (stack unwinding) и деструкторов.

RAII

Какие классы в стандартной библиотеке используют RAII

- `std::string`, `std::vector` и многие другие контейнеры/классы сами управляют памятью через RAII.

Задачи

1. 1512. Number of Good Pairs
   Дается массив интов, нужно вернуть количество хороших пар.
   Пара (i, j) называется хорошей, если nums[i] == nums[j] и i > j
   Input: nums = [1, 2, 3, 1, 1, 3];
   Output: 4
   Explanation: (0, 3), (0, 4), (3, 4), (2, 5)
2. 2114. Maximum Number of Words Found in Sentences
   Последовательность это список из слов, которые разделены пробелом.
   Вам дается массив из строк, где строка – это sentence[i].
   Верните максимальное количество слов, которые появляются в одном приложении.